PIC 10A 1A

TA: Bumsu Kim

This content is protected and may not be shared, uploaded, or distributed.

The author does not grant permission for these notes to be posted anywhere without prior consent.

UCLA

Today…

- HW2 Hints
- More Advanced/Challenging Exercises on Loops
  - Finding Matches
  - (Fibonacci/Prime Number Generator)
- (Pseudo-)Random Numbers in C++
  - Exercise: Random Walk Simulator

HW2 Hints

HW2 Hints

- Exercise 1: Consider three `if` clauses (you don't need `else`. why?)
- Exercise 2: Num2Rome → a bit more complicated
- But as we did in HW1, let's first think about a simplified problem:
  - `num` is between 1 and 9, inclusive, and we want to convert num into a Roman numeral
  - Four cases, basically:
    - `1 <= num < 4`: Print `I` `num` times
    - `num == 4`: Print `IV`.
    - `5 <= num < 9`: Print `V` first, and then `I` appropriate times (how many times?)
    - `num == 9`: Print `IX`.

  - If you can solve this simplified version, now you can extend this method.

UCLA

HW2 Hints

- Consider `num` between 1 and 99. There are two strategies:
  - A. Process ones digit first, and do `num/=10` (backward)
  - B. Process tens digit first, and do `num-=…` (forward)
- Either way is fine. For example, the pseudocode for strategy B:
  1. Ensure `num<100`
  2. Check if num has non-zero tens digit (`num>=10`?)
  3. Find the Roman numeral for this digit (using ‘X’, ‘L’, ‘C’)
  4. Subtract the corresponding number from num and check that `num<10`
  5. Similar to 2-4: check if `num>=1` and use ‘I’, ‘V’, ‘X’ to find the Roman numeral
- e.g. `num = 48`:
  - 1. and 2.: `num<100` and `num>=10`.
  - 3. and 4.: Find “XL” for 40, and `num-=40`
  - 5. `num>=1`, and find “VIII” for 8. `num-=8` and check if `num==0`

n	output
48	“”
8	“XL”
3	“XLV”
0	“XLVIII”

UCLA

HW2 Hints

- Strategy A (*backwards*): consider using the following operation:
  - `output = [something_to_add] + output;` // add a string *on the left*

- e.g. `num = 48`:
  - Take the ones digit `d = num%10`
  - Add the Roman numeral ("VIII") for `d` to the left
  - Throw ones digit away: `num/=10`
  - Compute `d = num%10` again
  - Add the Roman numeral ("XL") for `d` to the left

n	d	output
48	8	""
48	5	"V"
48	0	"VIII"
4	4	"VIII"
0	0	"XLVIII"

- This looks a bit more complicated, but in fact it's easier to "generalize"
  - See the next slides for this

UCLA

HW2 Hints: a smarter (but more difficult) method

- Challenge for Exercise 2:
- Can you solve this only using 3 if's and 2 while's?
- HINT: Consider strategy A, and the following helper variables:

```
string romans = "IVXLCDM"
int place = 0;
```

- And notice that
  - I, X, C, M → 0th, 2nd, 4th, and 6th, respectively
  - V, L, D → 1st, 3rd, and 5th, respectively
  - *i.e.*, in terms of the indices
    - 9: IX, XC, CM (02, 24, 46)
    - 4: IV, XL, CD (01, 23, 45)

UCLA

More Challenging Loop Exercises

Finding Matches

(Fibonacci/Prime Number Generator)

Finding Matches

- A code snippet finding the first match of the input character from the input string is given (see github repo)
- We will modify the program so that it finds
  - characters in case-**insensitive** manner (i.e. ‘h’ and ‘E’ are in “Hello, World!”)
  - **all** matches (i.e. prints out the position of every match)
    - ‘h’ in “Hello, World!”: 0th
    - ‘o’ in “Hello, World!”: 4th **and 8th**
    - ‘x’ in “Hello, World!”: none

UCLA

Fibonacci/Prime Number Generator

- 1. Fibonacci Number Generator
  - The user inputs a positive integer (**n**)
  - The program outputs the **n**-th Fibonacci number F_n
  - Fibonacci numbers satisfy: $F_{n+2} = F_{n+1} + F_n$
    (c.f. rabbit population puzzle posed by Fibonacci)

- 2. Prime Number Generator
  - The user inputs a positive integer (**n**)
  - The program outputs the **n**-th prime number P_n
  - First few prime numbers are: 2, 3, 5, 7, 11, 13, …

- Input and Output format:

```
Please enter a positive integer.
5
The 5th Fibonacci number is 5.
The 5th prime number is 11.
```

UCLA

Hints

- Fibonacci numbers are relatively simple
  - Just use a for loop
  - Can be done using 3 variables

- Prime numbers is more difficult
  - Break the problem into smaller pieces
  - First create a variable `int cnt` that counts the number of primes so far
  - You can then use a while loop

```
while(cnt < n) { // we want the n-th prime…
      // increment the current number
      // check if the current number is prime
      // if so, increment cnt
      // if not, don't increment cnt
```

UCLA

Hints

- Primality test (smaller piece of the entire problem)
- An integer is not prime if it has a divisor greater than 1
- So, consider using a loop to check if it has a divisor
- *Mathematical fact: you only need to check if it has a divisor $\leq \sqrt{N}$ → how to implement this w/o using `std::sqrt()`?
- You can set a flag `bool isPrime` to store the primality test result

```
for(div = 2; div*div <= num; ++div ) {
      // check if div divides num
      // if so, set isPrime false and exit from this (inner) loop
      // if not, do nothing
```

Q: what should be the *default value* for `isPrime`?

UCLA

(Pseudo-)Random Numbers in C++

rand() <cstdlib>

- `rand()` generates a (pseudo-) random integer,
  - Between `0` and `RAND_MAX`
    - RAND_MAX depends on the library
    - But at least 0x7fff = $2^{15} - 1 = 32767$ on any stand library implementation
  - Using a "seed"

- Using the same seed, `rand()` will generate the same sequence of random numbers
  - More on this later

UCLA

Using `rand()` – Floating Point numbers

- Since it generates an `int` between `0` and `RAND_MAX`, you can generate a random number between `0` and `X` (inclusive) by

  - `rand_num = (rand()*1./RAND_MAX)*X;`

    Type Conversion

    Normalized to 0 ~ 1

  - Or,

  - `rand_num = (static_cast<double>(rand())/RAND_MAX)*X;`

    which is preferable

UCLA

Using `rand()` – Floating Point numbers

- If you want a real number between A and B,
  - The size of interval = (B – A)
  - Starts from A

- `rand_num = ((rand()*1./RAND_MAX)*(B-A) + A;`

Between 0 and (B-A)

UCLA

Using `rand()` – Integers

- For integers, we can use the % (mod/remainder) operator

- Random int between 0 and N (inclusive):
  - `n = rand()%(N+1)`
  - The remainder is always between 0 and N
    - Note that, there are **N+1** different integers in [0,N]

- Random int between A and B:

  = A + (random int between 0 and (B-A))
  - `n = A + rand() % (B-A+1)`

UCLA

A Seed for `rand()`

- The function `rand()` generates a sequence of random numbers using a seed
- The seed can be set by `srand(some_number);`
- Ex) `some_number` == 1 → `rand()` gives
  - 41
  - 18467
  - 6334
  - 26500
  - 19169
  - 15724 …

  - Exercise: Use rand() several times without srand() commands, and verify that you get the sequence above

UCLA

Seed for `rand()`

- With the same seed, you'll have the same sequence of random numbers

- So, to get a random-like numbers (pseudo-random numbers), use different seed every time you run the code

- Commonly used trick: use a ***current time*** as a seed
  - `time(nullptr)` in `<ctime>` library returns the current time in **seconds**, since *00:00, Jan 1 1970 UTC*
  - Ex) `time(nullptr) == 1643894411` when this slide was created

UCLA

Summary

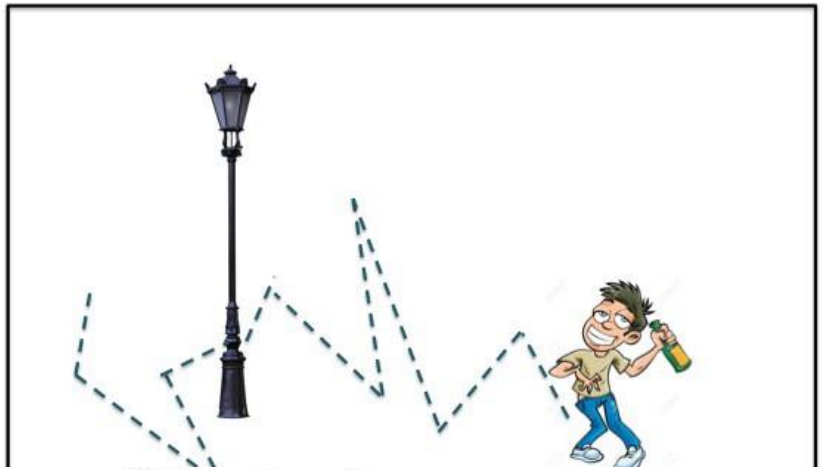
- #include `<cstdlib>` for `rand()` and `srand()`
- #include `<ctime>` for `time()`
- Use `time(nullptr)` to generate a seed:
  - `srand(time(nullptr));`
  - And then use `rand()`
- Intervals:
  - `r_double = (rand()*1./RAND_MAX)*(B-A) + A;`
  - `r_int = A + rand()%(B-A+1);`

UCLA

Exercise – Random Walk Simulator

(will be revisited on Thursday)

- Suppose someone walks in a constant speed, but chooses the direction randomly in each step
- The path (to be more precise, this *stochastic process*) is called a random walk



- We will simulate a 1-D random walk:
  - An object can only move forward or backward at each step
  - When the object hits the wall (upper/lower bounds), it stops walking
  - The time that it hits the wall is called the "stopping time"
  - Important in mathematical finance

Your Feedback is welcome

- Don’t hesitate to give a feedback on the discussion
- You can use a link in BruinLearn (Google Form)

Click this link

At this point, you may want to request
“more hw hints”,
“more challenging examples/exercises”,
“more details on the classroom examples”, etc.
Please let me know what you want!